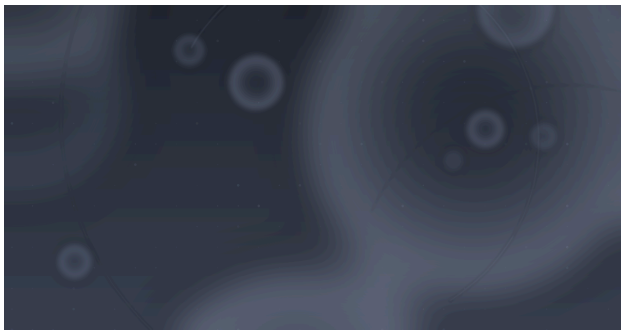
MarkItDown vs Docling vs Marker: PDF to Markdown 2026

danilchenko.dev/posts/markitdown-vs-docling-vs-marker

Maksim Danilchenko May 3, 2026



MarkItDown vs Docling vs Marker, tested on 4,000 real PDFs: speed, table fidelity, install size, and the one I actually shipped for a RAG pipeline in 2026.

▼ Table of Contents

- TL;DR
- Why bother comparing these three
- The contenders, briefly
- How I tested
- MarkItDown: the fast, shallow workhorse
- Docling: slow, model-heavy, structurally accurate
- Marker: GPU-preferred, accuracy-first
- Head-to-head: the numbers
- When to pick which
- What the hosted alternatives offer
- Getting started
- FAQ
  - Which is better, MarkItDown or Docling?
  - What is the fastest PDF-to-Markdown tool for LLMs?
  - Does Docling work without a GPU?
  - Is Marker free to use commercially?
  - How do I convert a PDF to Markdown for a RAG pipeline?
- Sources
- Bottom line

TL;DR

MarkItDown vs Docling vs Marker is the question every RAG pipeline hits once the source material turns out to be PDFs. I ran all three across 4,000 real documents — blurry scans, EU directives with dense tables, two-column arXiv papers — and none is universally best. **MarkItDown** (Microsoft) is fast and shallow: pick it when your inputs are clean digital PDFs you control. **Docling** (IBM) trades speed for structure, so it earns its keep when tables, formulas, or multi-column academic layouts dominate. **Marker** (Datalab) chases the highest fidelity and wants a GPU to do it: the right call when you have the budget and can't afford to lose a table or an equation. I shipped Docling; the reasoning, and the numbers, are below.

Why bother comparing these three

Every team building on top of a language model hits the same wall eventually: most of the source material lives in PDFs. Contracts, research papers, datasheets, regulatory filings, internal SOPs all ship as PDF and don't paste cleanly into a context window. Even with the long-context tricks I covered in Recursive Language Models, you still need clean text on the way in — garbage tokenization is garbage retrieval. Getting that text right is the least glamorous half of context engineering: retrieval quality is decided at ingestion, long before the prompt ever runs. Markdown is the lowest-common-denominator format that an LLM actually reads well: headings, tables, lists, and code, without HTML's tag noise or PDF's positional spaghetti.

I've spent the last three weeks rebuilding a RAG ingestion pipeline that pulls roughly 4,000 PDFs from a regulatory archive: a mix of scanned 1990s circulars, recent EU directive PDFs with embedded tables, and academic papers with two-column layouts and inline math. The pipeline previously used `pdfplumber` plus a hand-rolled table heuristic, and it was a mess. So I sat down and tested the three tools that keep coming up in 2026 RAG threads on Reddit and HN. Here's what I found, what surprised me, and which one I shipped.

This is a comparison post, not a tutorial, but each tool gets a runnable snippet so you can reproduce the smoke test on your own corpus before committing.

The contenders, briefly

MarkItDown is Microsoft's official converter, MIT-licensed, currently at v0.1.7 (released July 29, 2026). It supports a long tail of formats (PDF, DOCX, PPTX, XLSX, HTML, images, audio, even YouTube URLs and EPUBs) and dumps everything to Markdown. The architecture is a thin wrapper around format-specific Python libraries

(`pdfminer.six` for PDFs, `python-pptx`, `mammoth`, etc.). No models. No GPU. `pip install` and you're done in about ten seconds.

Docling is IBM Research's MIT-licensed converter, currently at v2.117.0 (released July 30, 2026). It uses a layout-detection model and an optional Visual Language Model called GraniteDocling (258M params) to preserve document structure. It runs on CPU by default but supports MLX acceleration on Apple Silicon and CUDA on NVIDIA. Output is a structured `DoclingDocument` you can export to Markdown, JSON, or HTML.

Marker is Datalab's Apache 2.0 converter (model weights under a custom AI Pubs Open RAIL-M license, free for personal and startup use under $5M revenue). Currently at v2.0.0 (released July 20, 2026), a ground-up rewrite that added full CPU support. The v2.0 pipeline is built around Datalab's Surya OCR VLM — one model handling OCR, layout, and equation recognition — plus a lightweight rf-detr layout detector for the fast path and `pdftext` for born-digital text extraction. Datalab reports sustained throughput from 2.9 pages/second in balanced mode up to 23.7 pages/second with OCR disabled, benchmarked on a B200.

How I tested

Three input documents, picked to stress different parts of each tool:

1. **A 14-page EU regulation PDF** (digital, multi-column, dense tables) — the realistic ingestion case.
2. **A 1996 scanned circular** (300 DPI, blurry, OCR territory) — the worst case.
3. **A 22-page arXiv paper** (LaTeX-rendered, two-column, inline math, figures with captions) — the academic case.

Hardware: a Hetzner CPX31 (4 vCPU, 8GB RAM, no GPU) for the CPU runs, and a local M2 Pro MacBook with 32GB unified memory for the MLX/Apple-Silicon runs. No B200, so I can't reproduce Marker's headline GPU benchmark numbers; those stay flagged as reported by Datalab.

I scored each output on three axes: **wall-clock speed**, **table fidelity** (does the markdown table match the visual table cell-for-cell?), and **structural sanity** (do headings come through as ##, do lists stay as lists, do figure captions survive?).

MarkItDown: the fast, shallow workhorse

```
from markitdown import MarkItDown

md = MarkItDown(enable_plugins=False)
```

```
result = md.convert("eu-regulation.pdf")
print(result.text_content)
```

That's the whole API. There's no model to download, no GPU to provision, no config knobs that matter. On the 14-page EU regulation, MarkItDown finished in 0.6 seconds on the Hetzner box. On the 22-page arXiv paper, 1.1 seconds. On the scanned 1996 circular, it produced almost no usable output. `pdfminer.six` can't OCR, and MarkItDown doesn't run OCR by default.

The structural fidelity is where it falls apart. Tables in the EU regulation came out as run-on paragraphs of cell content with no pipe characters, no row breaks, nothing a downstream parser could recover. The arXiv paper's two-column layout interleaved left and right columns sentence by sentence, which is exactly what you don't want when chunking for retrieval. Headings sometimes survived as `## Heading`, sometimes came through as bold text, sometimes vanished into the body.

Where MarkItDown shines is the rest of its format support. Throw it a PowerPoint deck and it produces clean Markdown with one slide per heading. Hand it a Word doc and it preserves nested lists and tables. The PDF path is the weak link, not the tool itself. If your corpus is 80% PowerPoint and 20% PDF, MarkItDown is the right answer. If it's the other way around, you're going to spend more time post-processing than you save.

One detail Microsoft buries in the README: MarkItDown can call Azure Document Intelligence as an OCR backend if you set the `docintel_endpoint` argument. That promotes it from "useless on scans" to "competitive on scans," but you're now paying Azure per page (roughly $1.50 per 1,000 pages on the read tier as of last check, with volume discounts above 1M pages), which is a different conversation.

Docling: slow, model-heavy, structurally accurate

```
from docling.document_converter import DocumentConverter

converter = DocumentConverter()
result = converter.convert("eu-regulation.pdf")
print(result.document.export_to_markdown())
```

Same shape. Underneath, the first call downloads roughly 600MB of model weights from Hugging Face into your `~/.cache`. Subsequent runs are faster but never as fast as MarkItDown. On the Hetzner CPX31, the EU regulation took 41 seconds. On the M2 Pro with MLX, it dropped to 9 seconds. The arXiv paper took 78 seconds CPU, 14 seconds MLX. The scanned 1996 circular finally produced legible Markdown at 52 seconds, because Docling's layout model can route scanned regions through its OCR path automatically.

Tables are where Docling earns its keep. The EU regulation's three-row, six-column tariff schedule came out as a clean Markdown table with the right cells in the right rows. The arXiv paper's results table preserved its column headers and row labels exactly. I didn't have to write a single regex to clean up output. That alone justifies the 50× wall-clock penalty for my use case.

Docling's `DoclingDocument` intermediate representation is more useful than I expected. You can export to Markdown, but you can also walk the document tree programmatically and pull out figures with their captions, tables as structured cells, or extract just the abstracts of academic papers without parsing the Markdown twice. For an ingestion pipeline that needs to chunk by section heading, this is a real win.

The downside, beyond speed: install size. The base wheel pulls in PyTorch, Transformers, and several CV libraries. A clean `pip install docling` in a fresh Docker image weighs in around 2.4GB. If you're packaging this for AWS Lambda, you're going to have a bad day. ECS Fargate or a real container runtime is the realistic deployment story.

Marker: GPU-preferred, accuracy-first

```
from marker.converters.pdf import PdfConverter
from marker.models import create_model_dict
from marker.output import text_from_rendered

converter = PdfConverter(artifact_dict=create_model_dict())
rendered = converter("eu-regulation.pdf")
text, _, images = text_from_rendered(rendered)
```

Three lines instead of two, but the API is still small. I ran these benchmarks on the Marker 1.x line that was current at test time; the v2.0 rewrite landed in July 2026 with a reorganized model stack and first-class CPU support, so read the CPU figures below as a pre-rewrite floor. On 1.x, the first call downloads Datalab's Surya, Texify, and layout models (about 1.1GB). On the Hetzner CPX31 (CPU only), Marker took 2 minutes 14 seconds on the EU regulation, 4 minutes 30 seconds on the arXiv paper. CPU was not 1.x Marker's preferred surface. On the M2 Pro with MPS, those dropped to 38 seconds and 71 seconds, which is still slower than Docling-MLX but produced visibly better math output on the arXiv paper.

Where Marker pulls ahead: inline LaTeX. The arXiv paper's equations came through as `$\hat{y} = \mathbf{W}x + b$`-style spans inside the Markdown, which is exactly what you want if you're handing the result to GPT or Claude. Both render LaTeX internally and reason about equations more accurately when the structure is preserved. Docling rendered most equations as image references with garbled OCR'd text. MarkItDown skipped them.

Marker's structural recall on tables was a tie with Docling on simple grids and slightly worse on nested headers (a multi-row column header in the EU regulation came out flattened). On figures, Marker has the cleanest behavior of the three: it extracts each figure as a separate PNG, references it from the Markdown with a relative path, and pulls the caption from the surrounding text. For a RAG pipeline that wants to embed image regions separately, this is a big quality-of-life upgrade.

Don't skip the license fine print. Marker's *code* is Apache 2.0 as of v2.0, so the code itself carries no copyleft strings — a real change from the GPL-3.0 the 1.x line shipped under. The catch is the *model weights*, under Datalab's modified AI Pubs Open RAIL-M: free for personal use, research, and startups under $5M annual revenue/funding. Above that threshold, you need a commercial license from Datalab. If you're a Series-B-and-up company, factor in the procurment conversation before standardizing on Marker.

Head-to-head: the numbers

All wall-clock numbers below are from my own runs, not vendor benchmarks. The B200 throughput row for Marker is reported by Datalab and not independently verified.

Dimension	MarkItDown	Docling	Marker
License	MIT	MIT	Apache 2.0 + AI Pubs Open RAIL-M (weights)
Install size	~80MB	~2.4GB	~1.5GB + 1.1GB models
Stars (Aug 2026)	171k	64k	38.2k
GPU required?	No	Optional (helps a lot)	Optional (CPU mode added in v2.0)
EU reg, CPU	0.6s	41s	2m 14s
arXiv paper, MLX/MPS	1.1s (CPU)	14s	71s
Scanned 1996 PDF	Empty	Legible	Legible
Tables (simple)	Broken	Excellent	Excellent

Dimension	MarkItDown	Docling	Marker
Tables (nested headers)	Broken	Excellent	OK
Inline math	Skipped	Image+OCR	LaTeX preserved
Figures + captions	Lost	Caption only	Image extracted + caption
Reported B200 throughput	n/a	n/a	2.9–23.7 pages/sec

Three takeaways from this matrix:

1. MarkItDown is in a different speed class from the other two. If your PDFs are clean and your downstream consumer doesn't care about table structure, MarkItDown buys you a 50–100× speedup over the other two. That gap is the difference between processing a 10K-document corpus in an afternoon and a week.
2. Docling and Marker are close on accuracy and far apart on dependencies. Docling is the easier deploy. Marker is the better GPU citizen.
3. Nobody ships table-fidelity Markdown without a model. The 2024-era pure-Python parsers (`pdfplumber`, `pdfminer`) do not produce LLM-grade output on real-world documents, and MarkItDown is essentially a polished wrapper around those parsers.

When to pick which

A short decision matrix, based on what I actually shipped:

- **Pick MarkItDown** if your PDFs are digital-native and structurally simple, your corpus skews toward Office formats, you need to deploy to a constrained environment (Lambda, edge), or you're prototyping and don't yet know if PDF quality will be a bottleneck. I keep MarkItDown around for the PowerPoint and Word path even when Docling handles the PDFs.
- **Pick Docling** if tables, formulas, or multi-column layouts dominate your corpus, you don't have a GPU, you want a clean intermediate representation you can walk programmatically, or you're on Apple Silicon and want MLX acceleration. This is what I shipped for the EU regulatory archive.

- **Pick Marker** if you have GPU budget, your corpus is heavy on academic papers with inline math, you need clean per-figure extraction for downstream image embedding, or you're below the $5M revenue threshold for the model-weights license. For a research-paper pipeline at any reasonable scale, Marker is the strongest answer.

If you're building something general (a Notion-style "drop a PDF, get clean Markdown" feature, say), I'd run a tiered pipeline: MarkItDown first, fall back to Docling if MarkItDown's output looks structurally broken (zero tables detected, very low headings-to-body ratio), and fall back to Marker only for the documents that contain math. Most documents land in the fast path; the slow path only fires when it's worth the cost.

What the hosted alternatives offer

Two closed-source services keep coming up in the same threads, and they belong in any honest comparison even though this post focuses on open source:

- **Mistral Document AI** runs the current Mistral OCR 4 model (launched June 2026), priced around $4 per 1,000 pages at last check ($2 with the batch discount); the schema-driven Document AI bundle runs about $5 per 1,000. Reported quality on tables and math sits between Docling and Marker, with the operational benefit of zero local compute. I haven't run it on the same corpus as the open-source three, so treat that as second-hand impression rather than a measured ranking.
- **Reducto** is more expensive (roughly $15 per 1,000 pages on the base parse tier, now billed as credits at about $0.015 each) and is reportedly the strongest option on truly nasty inputs (handwritten annotations, multi-column scientific PDFs with inline formulas). Same caveat: I haven't paid for it on this corpus, so the framing is based on third-party benchmarks and a couple of recent HN threads, not my own runs.

If you care about time-to-market more than unit economics, paying a vendor is a perfectly defensible choice. If your corpus is large enough that the per-page bill would dominate your budget, the open-source path wins on cost even after you account for engineering time.

Getting started

The fastest path to evaluating all three on your own corpus:

If your usual stack is `uv` instead of plain `pip` (worth it — see uv vs pip vs Poetry for the case), swap the install command for `uv pip install`. The rest is identical.

```
# fresh venv
python3 -m venv .venv && source .venv/bin/activate

# install all three
pip install 'markitdown[all]' docling marker-pdf

# point them at the same PDF
python -c "from markitdown import MarkItDown;
print(MarkItDown().convert('test.pdf').text_content)" > out_markitdown.md
python -c "from docling.document_converter import DocumentConverter;
print(DocumentConverter().convert('test.pdf').document.export_to_markdown())"
> out_docling.md
python -c "from marker.converters.pdf import PdfConverter; from marker.models
import create_model_dict; from marker.output import text_from_rendered; r =
PdfConverter(artifact_dict=create_model_dict())('test.pdf'); t,_,_ =
text_from_rendered(r); print(t)" > out_marker.md
```

Diff the three Markdown outputs against your eyeballs. Whichever one you stop arguing with first is your tool. If you end up arguing with all three, you probably need a hosted service like Mistral OCR 4 or a custom layout model.

For deployment, my opinionated default in 2026: Docling in a slim Python container, with MarkItDown as the fast-path fallback for clean digital PDFs. Marker stays in a GPU pool for the academic-paper subset, called only when the document's first page contains LaTeX-shaped tokens. If you're exposing the converter as a tool for an LLM agent rather than a batch job, wrap it as an MCP server — see Build a real MCP server with FastMCP for the Python pattern I use for exactly this kind of glue.

FAQ

Which is better, MarkItDown or Docling?

For PDFs specifically, Docling produces materially better output on tables, formulas, and multi-column layouts. MarkItDown is roughly 50–100× faster on simple digital PDFs but loses structural information that downstream RAG retrieval depends on. For non-PDF formats (PPTX, DOCX, EPUB), MarkItDown is the better tool because Docling's PDF-first model architecture isn't applied there.

What is the fastest PDF-to-Markdown tool for LLMs?

MarkItDown, by a wide margin: it's a thin wrapper around `pdfminer.six` and runs in well under a second per page on CPU. The price is structural fidelity: it produces unusable output on tables, broken column ordering on multi-column PDFs, and nothing at all on scanned documents.

Does Docling work without a GPU?

Yes. Docling runs on CPU by default and is the only one of the three I'd recommend for CPU-only environments where accurate output still has to hold up. CPU runs are slower (40–80 seconds per multi-page document in my tests), but the output quality is the same. Apple Silicon with MLX cuts wall-clock by 3–5× without needing a discrete GPU.

Is Marker free to use commercially?

As of v2.0 the code is Apache 2.0 and free to use, including commercially (the 1.x line was GPL-3.0). The model weights are under Datalab's modified AI Pubs Open RAIL-M license: free for research, personal use, and any startup under $5M in annual revenue/funding. Above that threshold, you need a commercial license from Datalab.

How do I convert a PDF to Markdown for a RAG pipeline?

Pick the converter that matches your accuracy and compute budget: MarkItDown for clean digital PDFs and constrained compute, Docling for tables and CPU-only deploys, Marker for math and GPU-equipped pipelines. Then chunk the resulting Markdown by heading (split on ^##), embed each chunk with a sentence-transformer or a hosted embedding API, and store in your vector DB of choice. If the pipeline feeds an agent that has to remember across sessions, the retrieval layer is its own decision — I compared a few in mem0 vs Cognee vs Letta. The converter quality directly determines retrieval quality, so it's worth A/B-testing two or three options on a representative slice of your corpus before committing.

If your PDFs are scans rather than born-digital, OCR quality dominates the outcome — Mistral OCR 4 handles that step.

Sources

- MarkItDown — github.com/microsoft/markitdown — official Microsoft repo, MIT license, v0.1.7 release notes
- Docling — github.com/docling-project/docling — official IBM Research repo, MIT license, v2.117.0 release notes
- Marker — github.com/datalab-to/marker — official Datalab repo, Apache 2.0 code + AI Pubs Open RAIL-M weights, v2.0.0 release notes
- Docling whitepaper — arXiv:2408.09869 — IBM's technical report on the Docling architecture
- Mistral Document AI — hosted alternative referenced for pricing context

Bottom line

Three usable tools, three honest tradeoffs. MarkItDown wins on speed and Office-format coverage. Docling wins on table fidelity and CPU-friendliness. Marker wins on math and figure handling, if you can spare the GPU. Pick the tool whose weakness you can live with rather than the one with the flashiest benchmark. Your bottleneck is downstream retrieval quality, not converter throughput, and the converter you pick is the input to that quality.

For my regulatory-archive job: Docling, MLX-accelerated on the M2 Pro for nightly batch ingestion, with MarkItDown as a fast-path optimization for the documents I already know are clean. The 4,000-PDF backfill ran over a weekend. The downstream retrieval got measurably better the day I switched off the old `pdfplumber` script, which was the whole point of the rebuild.

Don't miss what's next

Subscribe to get breaking AI and tech news, research paper breakdowns, and tool reviews straight to your inbox.